

✓ MSFT Forecasting: ARIMA, VECM and GARCH

DA4421 Time Series Econometrics take-home assignment. Daily MSFT closing prices (April 2015 to April 2026) with S&P 500 and VIX as extra variables.

✓ PART 1: Univariate ARIMA(p,d,q) on Daily Data

✓ Load, Clean, and Explore Data

```

1 import pandas as pd
2 import numpy as np
3 import yfinance as yf
4 import matplotlib.pyplot as plt
5 from datetime import datetime
6 import warnings
7 warnings.filterwarnings('ignore')
8
9 # Download data
10 tickers = ['MSFT', '^GSPC', '^VIX']
11 data = yf.download(tickers, start='2015-04-01', end='2026-05-01', progress=False)['Close']
12 data.columns = ['MSFT', 'SP500', 'VIX']
13
14 # Use daily data
15 df = data.copy()
16
17 print(df.head())
18 print(df.info())
19 print(df.describe())
20 print(f"From {df.index.min()} to {df.index.max()}")
21 print(f"Missing values: {df.isna().sum()}")
22
23 # Save
24 df.to_csv('MSFT_Multivariate_Data.csv')
25

```

```

      MSFT      SP500      VIX
Date
2015-04-01  34.736565  2059.689941  15.11
2015-04-02  34.369770  2066.959961  14.67
2015-04-06  35.444607  2080.620117  14.74
2015-04-07  35.427551  2076.330078  14.78
2015-04-08  35.333706  2081.899902  13.98
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 2787 entries, 2015-04-01 to 2026-04-30
Data columns (total 3 columns):
#   Column  Non-Null Count  Dtype
---  ---
0   MSFT    2787 non-null    float64
1   SP500   2787 non-null    float64
2   VIX     2787 non-null    float64
dtypes: float64(3)
memory usage: 87.1 KB
None
      MSFT      SP500      VIX
count  2787.000000  2787.000000  2787.000000
mean    214.649375  3743.954398   18.428281
std     143.901486  1410.020410    7.130172
min      34.369770  1829.079956    9.140000
25%     79.493378  2588.050049   13.595000
50%    202.800964  3443.439941   16.660000
75%    322.918488  4529.600098   21.375000
max     537.646423  7209.009766   82.690002
From 2015-04-01 00:00:00 to 2026-04-30 00:00:00
Missing values: MSFT      0
SP500      0
VIX      0
dtype: int64

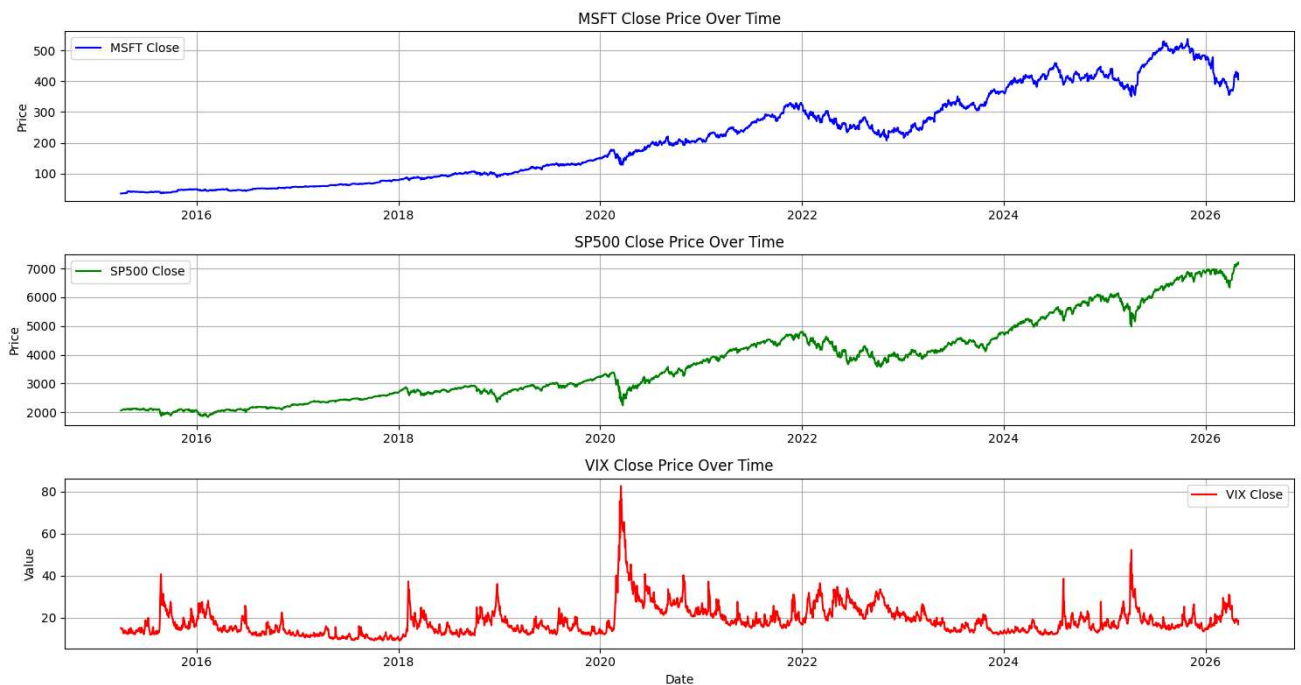
```

Visualization of Time Series Data

```

1 plt.figure(figsize=(15, 8))
2
3 plt.subplot(3, 1, 1) # 3 rows, 1 column, 1st plot
4 plt.plot(df.index, df['MSFT'], label='MSFT Close', color='blue')
5 plt.title('MSFT Close Price Over Time')
6 plt.ylabel('Price')
7 plt.grid(True)
8 plt.legend()
9
10 plt.subplot(3, 1, 2) # 3 rows, 1 column, 2nd plot
11 plt.plot(df.index, df['SP500'], label='SP500 Close', color='green')
12 plt.title('SP500 Close Price Over Time')
13 plt.ylabel('Price')
14 plt.grid(True)
15 plt.legend()
16
17 plt.subplot(3, 1, 3) # 3 rows, 1 column, 3rd plot
18 plt.plot(df.index, df['VIX'], label='VIX Close', color='red')
19 plt.title('VIX Close Price Over Time')
20 plt.xlabel('Date')
21 plt.ylabel('Value')
22 plt.grid(True)
23 plt.legend()
24
25 plt.tight_layout()
26 plt.show()

```



We use Microsoft (MSFT) daily closing prices from April 2015 to April 2026 (2,787 observations).

The series displays a strong upward trend, especially after 2020, and shows a sharp drop during the COVID-19 crash in March 2020, visible in the structural-break plot.

VIX and S&P 500 are also downloaded for later use in a multivariate model.

Test for Stationarity & Decide Differencing (d)

```

1 from statsmodels.tsa.stattools import adfuller, kpss
2

```

```

3 def stationarity_tests(series, title=""):
4     print(f"\n=== {title} ===")
5     adf = adfuller(series, autolag='AIC')
6     print(f'ADF Statistic: {adf[0]:.6f}, p-value: {adf[1]:.6f}')
7     kpss_stat, kpss_p, _, _ = kpss(series, regression='c')
8     print(f'KPSS Statistic: {kpss_stat:.6f}, p-value: {kpss_p:.6f}')
9
10
11 stationarity_tests(df['MSFT'], "Original MSFT Series")
12 stationarity_tests(df['MSFT'].diff().dropna(), "First Difference of MSFT Series")
13
14

```

=== Original MSFT Series ===

ADF Statistic: -0.609983, p-value: 0.868710

KPSS Statistic: 8.368368, p-value: 0.010000

=== First Difference of MSFT Series ===

/tmp/ipykernel_576/143962386.py:7: InterpolationWarning: The test statistic is outside of the range of p-values available
 kpss_stat, kpss_p, _, _ = kpss(series, regression='c')

ADF Statistic: -17.524786, p-value: 0.000000

KPSS Statistic: 0.053690, p-value: 0.100000

/tmp/ipykernel_576/143962386.py:7: InterpolationWarning: The test statistic is outside of the range of p-values available
 kpss_stat, kpss_p, _, _ = kpss(series, regression='c')

ADF test on levels: p-value = 0.87 → fail to reject unit root.

KPSS test: rejects stationarity.

After first differencing, the ADF p-value is ~0 and the KPSS p-value is 0.10, indicating the differenced series is stationary.

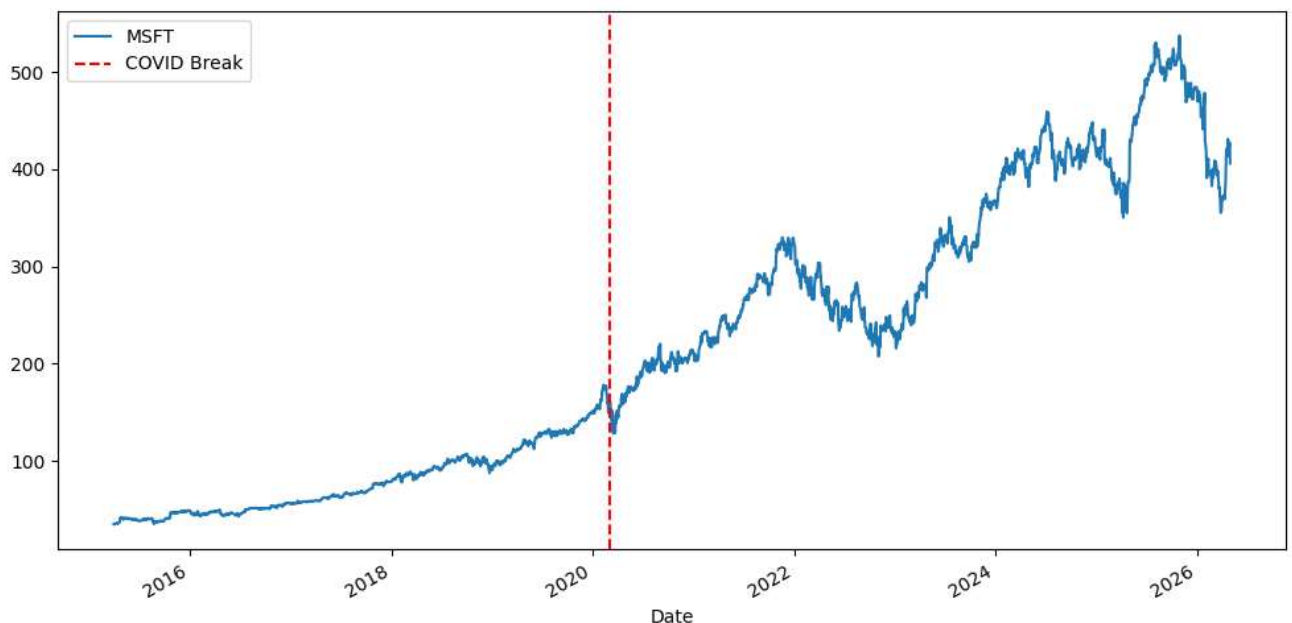
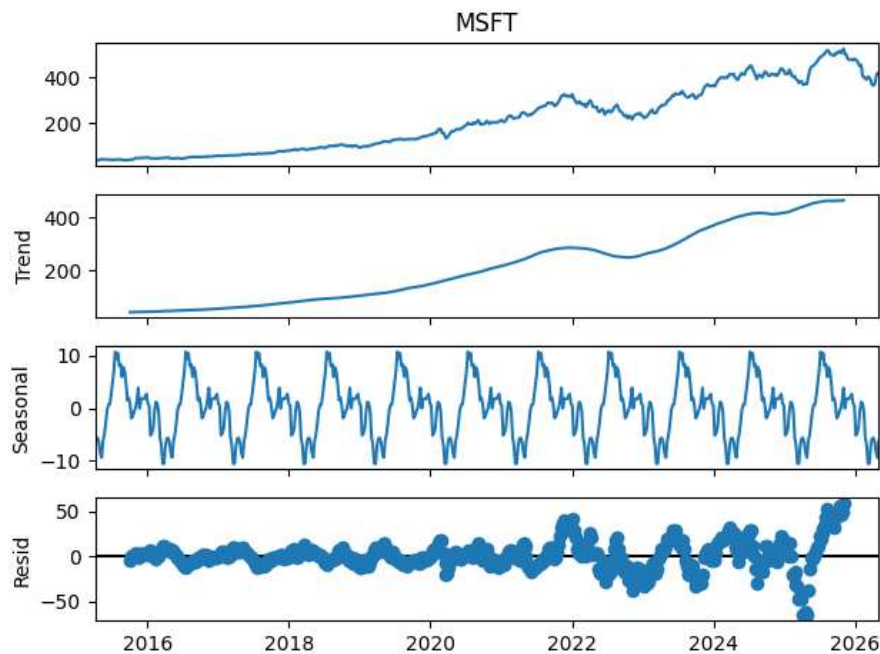
Conclusion: MSFT daily prices are I(1) → we set d=1.

✓ Check Seasonality & Structural Breaks

```

1 # Seasonal decomposition (additive/multiplicative)
2 from statsmodels.tsa.seasonal import seasonal_decompose
3 decomp = seasonal_decompose(df['MSFT'].resample('W').mean(), model='additive', period=52) # weekly
4 decomp.plot()
5 plt.show()
6
7 # Structural breaks (simple Chow or visual)
8 plt.figure(figsize=(12,6))
9 df['MSFT'].plot()
10 plt.axvline('2020-03-01', color='red', linestyle='--', label='COVID Break')
11 plt.legend()
12 plt.show()

```



- ✓ ACF/PACF for p, q + Auto ARIMA
- ✓ Plot ACF and PACF of first differenced series

```
1 import sys
2 !{sys.executable} -m pip install pmdarima
```

```
Collecting pmdarima
  Downloading pmdarima-2.1.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata
Requirement already satisfied: joblib>=0.11 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (1.6.0)
Requirement already satisfied: Cython!=0.29.18,!=0.29.31,>=0.29 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (0.29.37)
Requirement already satisfied: numpy>=1.21.6 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (2.1.3)
Requirement already satisfied: pandas>=0.19 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (2.2.3)
Requirement already satisfied: scikit-learn>=0.22 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (1.6.1)
Requirement already satisfied: scipy>=1.13.0 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (1.16.3)
Requirement already satisfied: statsmodels>=0.14.5 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (0.15.0)
Requirement already satisfied: urllib3 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (2.5.0)
Requirement already satisfied: setuptools!=50.0.0,>=42 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (80.1.0)
Requirement already satisfied: packaging>=17.1 in /usr/local/lib/python3.13/dist-packages (from pmdarima) (26.3)
```

Requirement already satisfied: cloudpickle>=3.0 in /usr/local/lib/python3.13/dist-packages (from joblib>=0.11->pmdarima)

Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/lib/python3.13/dist-packages (from pandas>=0.19->pmdarima)

Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.13/dist-packages (from pandas>=0.19->pmdarima)

Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.13/dist-packages (from pandas>=0.19->pmdarima)

Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.13/dist-packages (from scikit-learn>=0.22->pmdarima)

Requirement already satisfied: patsy>=0.5.6 in /usr/local/lib/python3.13/dist-packages (from statsmodels>=0.14.5->pmdarima)

Requirement already satisfied: formulaic>=1.1.0 in /usr/local/lib/python3.13/dist-packages (from statsmodels>=0.14.5->pmdarima)

Requirement already satisfied: interface-meta>=1.2.0 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmodels)

Requirement already satisfied: narwhals>=1.17 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmodels)

Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmodels)

Requirement already satisfied: wrapt>=1.17.0rc1 in /usr/local/lib/python3.13/dist-packages (from formulaic>=1.1.0->statsmodels)

Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.13/dist-packages (from python-dateutil>=2.8.2->pandas)

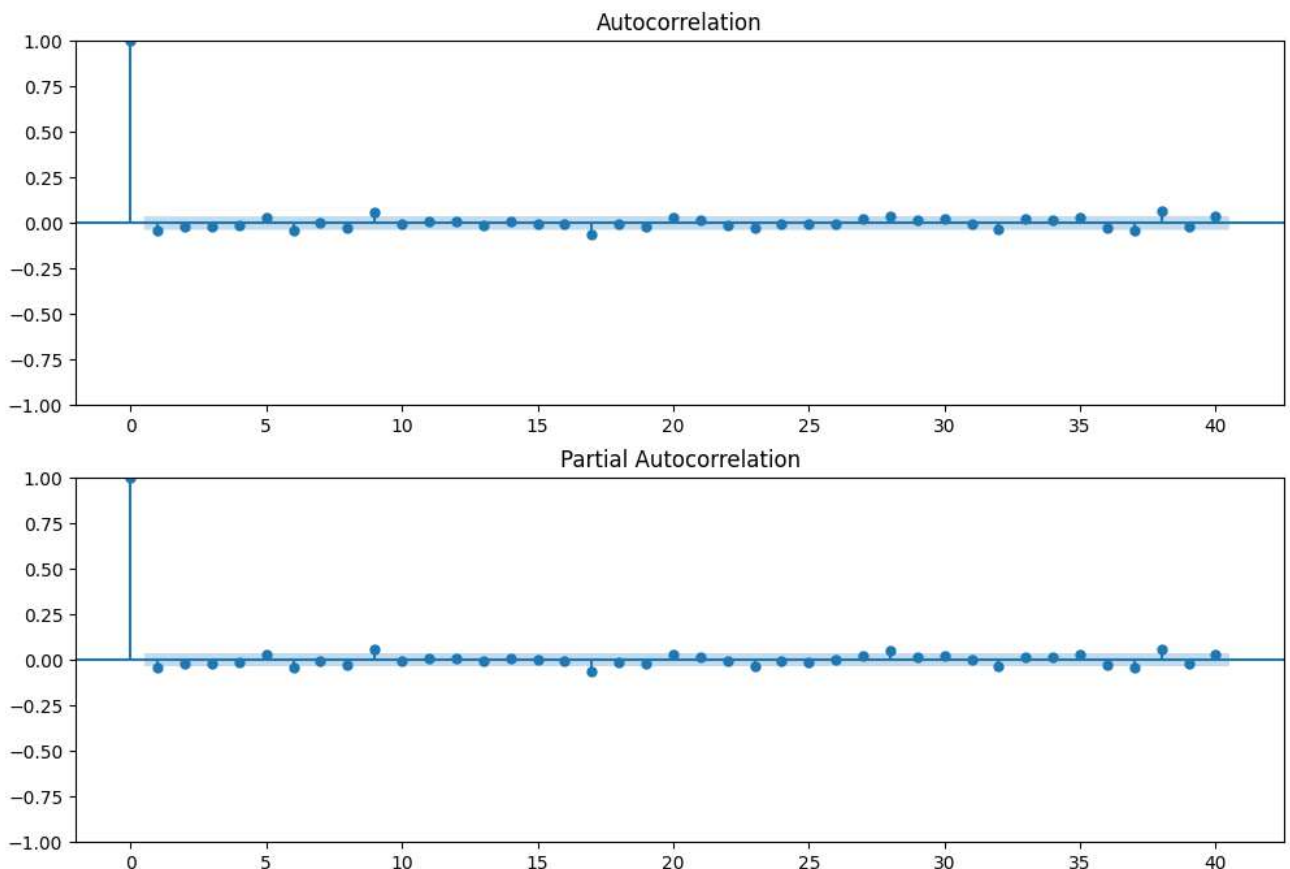
Downloading pmdarima-2.1.1-cp313-cp313-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (688 kB)

688.9/688.9 kB 9.1 MB/s eta 0:00:00

Installing collected packages: pmdarima

Successfully installed pmdarima-2.1.1

```
1 from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
2 import pmdarima as pm
3
4 # Plot ACF and PACF of first differenced series
5 fig, axes = plt.subplots(2,1, figsize=(12,8))
6 plot_acf(df['MSFT'].diff().dropna(), lags=40, ax=axes[0])
7 plot_pacf(df['MSFT'].diff().dropna(), lags=40, ax=axes[1])
8 plt.show()
9
10
```



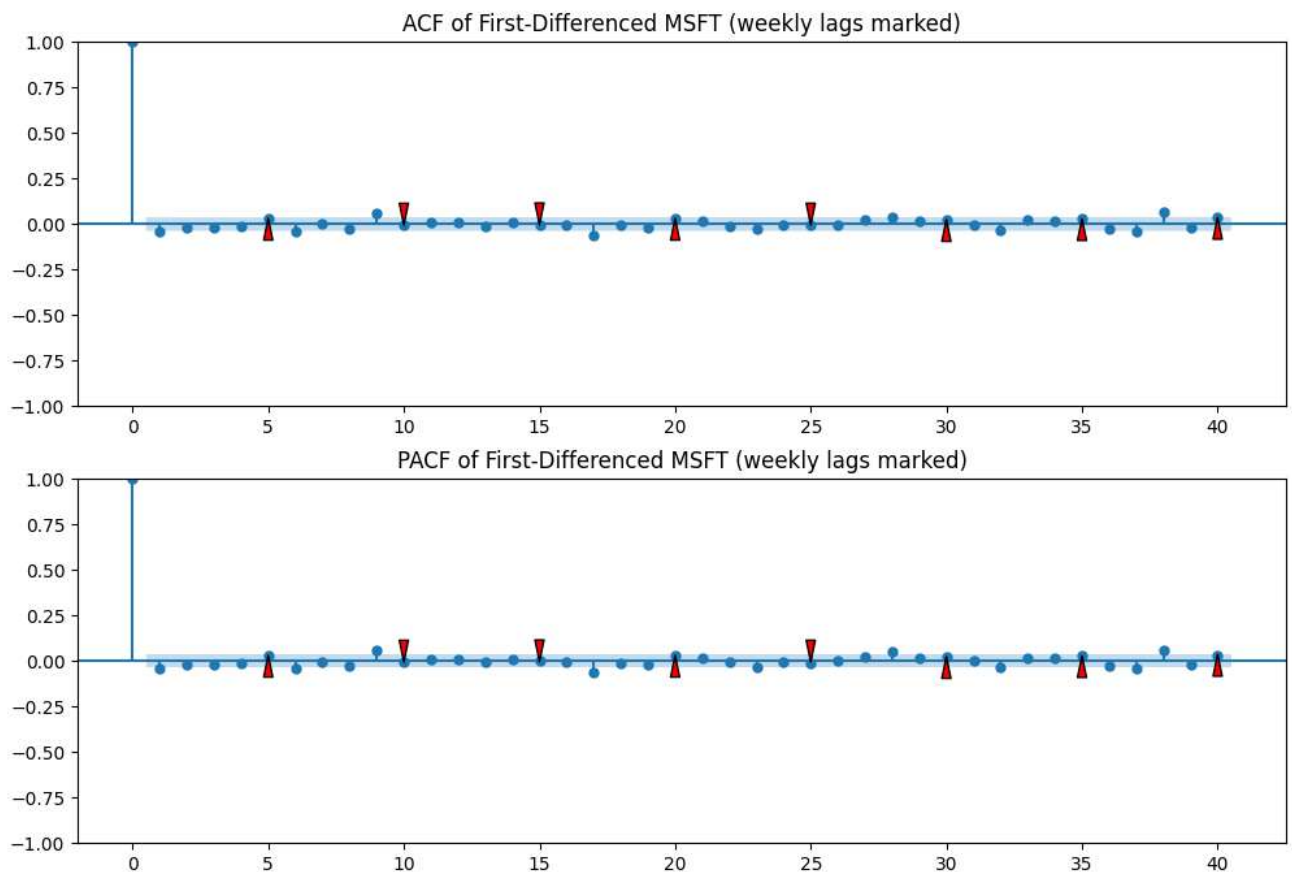
✦ ACF/PACF with weekly lag markers

```
1 fig, axes = plt.subplots(2,1, figsize=(12,8))
2 diff = df['MSFT'].diff().dropna()
3
4 plot_acf(diff, lags=40, ax=axes[0])
5 axes[0].set_title("ACF of First-Differenced MSFT (weekly lags marked)")
6 for lag in [5,10,15,20,25,30,35,40]:
```

```

7 axes[0].annotate('', xy=(lag, diff.autocorr(lag)), xytext=(lag, 0.02),
8                 arrowprops=dict(facecolor='red', shrink=0.05, width=1, headwidth=5))
9
10 plot_pacf(diff, lags=40, ax=axes[1])
11 axes[1].set_title("PACF of First-Differenced MSFT (weekly lags marked)")
12 for lag in [5,10,15,20,25,30,35,40]:
13     axes[1].annotate('', xy=(lag, diff.autocorr(lag)), xytext=(lag, 0.02),
14                     arrowprops=dict(facecolor='red', shrink=0.05, width=1, headwidth=5))
15 plt.show()

```



✓ Auto ARIMA

```

1 import pmdarima as pm
2 from statsmodels.tsa.arima.model import ARIMA
3
4 # Auto ARIMA
5 model = pm.auto_arima(df['MSFT'],
6                       start_p=0, start_q=0,
7                       max_p=5, max_q=5,
8                       d=1, # We already know d=1
9                       seasonal=True, m=5, # or try m=21 (trading days)
10                      trace=True,
11                      error_action='ignore',
12                      suppress_warnings=True)
13
14 print(model.summary())
15
16 # Compare SARIMA vs. ARIMA (non-seasonal)
17 ts = df['MSFT'] # Define ts for the ARIMA model
18 non_seasonal = ARIMA(ts, order=(1,1,1)).fit()
19
20 print("=== Model Comparison ===")
21 print(f"Non-seasonal ARIMA(1,1,1) AIC: {non_seasonal.aic:.2f}, BIC: {non_seasonal.bic:.2f}")
22 print(f"Seasonal SARIMA(1,1,1)(0,0,1)_5 AIC: {model.aic:.2f}, BIC: {model.bic:.2f}")

```

```

ARIMA(0,1,1)(0,0,2)[5] intercept : AIC=15804.158, Time=4.83 sec
ARIMA(0,1,1)(1,0,0)[5] intercept : AIC=15802.197, Time=2.00 sec
ARIMA(0,1,1)(1,0,2)[5] intercept : AIC=15806.153, Time=7.06 sec
ARIMA(0,1,0)(0,0,1)[5] intercept : AIC=15805.339, Time=1.08 sec
ARIMA(1,1,1)(0,0,1)[5] intercept : AIC=15801.169, Time=2.48 sec
ARIMA(1,1,1)(0,0,0)[5] intercept : AIC=15802.890, Time=1.04 sec
ARIMA(1,1,1)(1,0,1)[5] intercept : AIC=15803.163, Time=3.00 sec
ARIMA(1,1,1)(0,0,2)[5] intercept : AIC=15803.160, Time=6.48 sec
ARIMA(1,1,1)(1,0,0)[5] intercept : AIC=15801.197, Time=2.13 sec
ARIMA(1,1,1)(1,0,2)[5] intercept : AIC=15805.154, Time=5.52 sec
ARIMA(1,1,0)(0,0,1)[5] intercept : AIC=15802.362, Time=1.49 sec
ARIMA(2,1,1)(0,0,1)[5] intercept : AIC=15802.957, Time=4.97 sec
ARIMA(1,1,2)(0,0,1)[5] intercept : AIC=15802.962, Time=4.18 sec
ARIMA(0,1,2)(0,0,1)[5] intercept : AIC=15803.315, Time=1.80 sec
ARIMA(2,1,0)(0,0,1)[5] intercept : AIC=15803.482, Time=1.32 sec
ARIMA(2,1,2)(0,0,1)[5] intercept : AIC=15805.154, Time=5.63 sec
ARIMA(1,1,1)(0,0,1)[5] intercept : AIC=15802.733, Time=0.91 sec

```

Best model: ARIMA(1,1,1)(0,0,1)[5] intercept

Total fit time: 79.184 seconds

SARIMAX Results

```

=====
Dep. Variable:          y      No. Observations:      2787
Model:          SARIMAX(1, 1, 1)x(0, 0, 1, 5)      Log Likelihood      -7895.585
Date:              Thu, 08 Oct 2026      AIC      15801.169
Time:              08:44:11      BIC      15830.831
Sample:              0      HQIC      15811.879
              - 2787
Covariance Type:      opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
intercept	0.0390	0.028	1.396	0.163	-0.016	0.094
ar.L1	0.7093	0.120	5.905	0.000	0.474	0.945
ma.L1	-0.7480	0.114	-6.585	0.000	-0.971	-0.525
ma.S.L5	0.0409	0.012	3.493	0.000	0.018	0.064
sigma2	16.9481	0.179	94.805	0.000	16.598	17.299

```

=====
Ljung-Box (L1) (Q):      0.07      Jarque-Bera (JB):      19285.05
Prob(Q):      0.79      Prob(JB):      0.00
Heteroskedasticity (H):      36.14      Skew:      -0.58
Prob(H) (two-sided):      0.00      Kurtosis:      15.84
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been pr
self._init_dates(dates, freq)

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been pr
self._init_dates(dates, freq)

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been pr
self._init_dates(dates, freq)

=== Model Comparison ===

Non-seasonal ARIMA(1,1,1) AIC: 15804.42, BIC: 15822.21

Seasonal SARIMA(1,1,1)(0,0,1)_5 AIC: 15801.17, BIC: 15830.83

Model Identification: The auto_arima function has successfully identified an ARIMA(1,1,1) model with a seasonal component (0,0,1,5) which minimizes the AIC

Residual Autocorrelation: The Ljung-Box test shows a high p-value (0.79). There's no significant autocorrelation left in model's residuals, indicating that the model has captured the time-dependent patterns effectively.

Fit, Diagnose, Forecast

```

1 from statsmodels.tsa.arima.model import ARIMA
2 from statsmodels.tsa.stattools import acf
3
4 # Extract order and seasonal_order from the auto_arima model
5 p, d, q = model.order
6 P, D, Q, s = model.seasonal_order
7
8 # Define the time series
9 ts = df['MSFT']

```

```
10
11 arima_model = ARIMA(ts, order=(p,d,q), seasonal_order=(P,D,Q,s)).fit()
12 print(arima_model.summary())
13
14 # Diagnostics
15 arima_model.plot_diagnostics(figsize=(12,8))
16 plt.show()
17
18 # Residual tests using correct Ljung-Box
19 from statsmodels.stats.diagnostic import acorr_ljungbox
20 lb_test = acorr_ljungbox(arima_model.resid, lags=[10,20,30], return_df=True)
21 print("Ljung-Box test results (p-values):")
22 print(lb_test)
```

```

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been pr
self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been pr
self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been pr
self._init_dates(dates, freq)

```

SARIMAX Results

```

=====
Dep. Variable:          MSFT      No. Observations:      2787
Model:                ARIMA(1, 1, 1)x(0, 0, 1, 5)      Log Likelihood      -7897.367
Date:                  Thu, 08 Oct 2026              AIC              15802.733
Time:                  08:44:13                     BIC              15826.463
Sample:                0                            HQIC             15811.301
                        - 2787
Covariance Type:       opg
=====

```

	coef	std err	z	P> z	[0.025	0.975]
ar.L1	0.6830	0.140	4.895	0.000	0.410	0.957
ma.L1	-0.7210	0.133	-5.427	0.000	-0.981	-0.461
ma.S.L5	0.0404	0.012	3.378	0.001	0.017	0.064
sigma2	16.9698	0.178	95.390	0.000	16.621	17.319

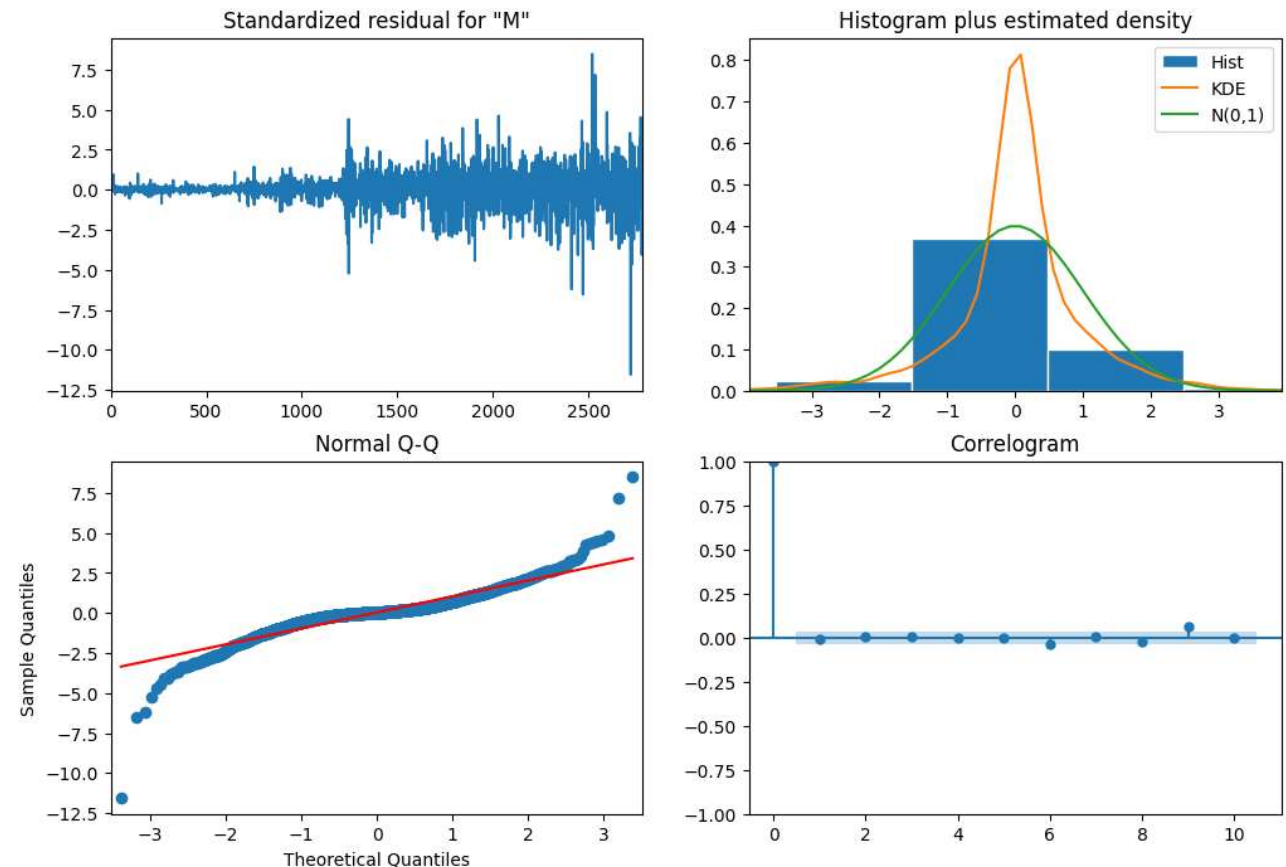
```

=====
Ljung-Box (L1) (Q):          0.10      Jarque-Bera (JB):          19356.44
Prob(Q):                    0.76      Prob(JB):                0.00
Heteroskedasticity (H):     36.14     Skew:                    -0.57
Prob(H) (two-sided):        0.00     Kurtosis:                 15.86
=====

```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).



Ljung-Box test results (p-values):

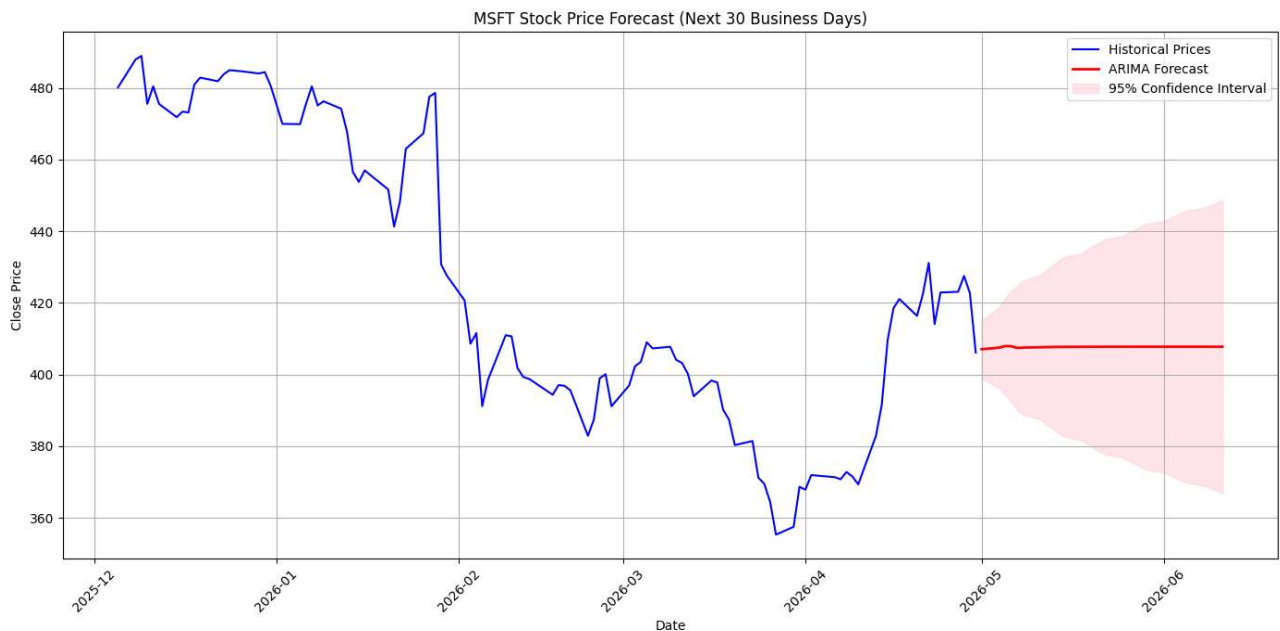
	lb_stat	lb_pvalue
10	15.134219	0.127245
20	30.192077	0.066801
30	43.178251	0.056508

```

1 forecast_steps = 30
2
3 # Future business days. Passing this index to get_forecast avoids the
4 # "No supported index is available" error in newer statsmodels versions.
5 future_dates = pd.date_range(start=df.index[-1] + pd.Timedelta(days=1),
6                               periods=forecast_steps, freq='B')
7
8 forecast = arima_model.get_forecast(steps=forecast_steps, index=future_dates)
9 forecast_mean = forecast.predicted_mean
10 conf_int = forecast.conf_int(alpha=0.05)
11
12 forecast_df = pd.DataFrame({
13     'Forecast': forecast_mean.values,
14     'Lower_CI': conf_int.iloc[:,0].values,
15     'Upper_CI': conf_int.iloc[:,1].values
16 }, index=future_dates)
17
18 print(forecast_df.round(2))
19
20 # Plot
21 plt.figure(figsize=(14, 7))
22 plt.plot(df['MSFT'][-100:], label='Historical Prices', color='blue')
23 plt.plot(forecast_df['Forecast'], label='ARIMA Forecast', color='red', linewidth=2)
24 plt.fill_between(forecast_df.index, forecast_df['Lower_CI'], forecast_df['Upper_CI'],
25                 color='pink', alpha=0.4, label='95% Confidence Interval')
26 plt.title('MSFT Stock Price Forecast (Next 30 Business Days)')
27 plt.xlabel('Date')
28 plt.ylabel('Close Price')
29 plt.legend()
30 plt.grid(True)
31 plt.xticks(rotation=45)
32 plt.tight_layout()
33 plt.show()

```

	Forecast	Lower_CI	Upper_CI
2026-05-01	407.08	399.01	415.15
2026-05-04	407.50	396.30	418.71
2026-05-05	407.95	394.43	421.46
2026-05-06	407.93	392.52	423.35
2026-05-07	407.42	390.36	424.48
2026-05-08	407.53	388.86	426.20
2026-05-11	407.61	387.48	427.74
2026-05-12	407.66	386.19	429.13
2026-05-13	407.70	384.97	430.43
2026-05-14	407.72	383.81	431.64
2026-05-15	407.74	382.70	432.78
2026-05-18	407.75	381.64	433.86
2026-05-19	407.76	380.61	434.90
2026-05-20	407.76	379.63	435.90
2026-05-21	407.77	378.67	436.86
2026-05-22	407.77	377.75	437.79
2026-05-25	407.77	376.86	438.69
2026-05-26	407.77	375.98	439.56
2026-05-27	407.77	375.14	440.41
2026-05-28	407.77	374.31	441.24
2026-05-29	407.77	373.50	442.05
2026-06-01	407.77	372.71	442.83
2026-06-02	407.77	371.94	443.61
2026-06-03	407.77	371.19	444.36
2026-06-04	407.77	370.45	445.10
2026-06-05	407.77	369.72	445.83
2026-06-08	407.77	369.01	446.54
2026-06-09	407.77	368.31	447.24
2026-06-10	407.77	367.63	447.92
2026-06-11	407.77	366.95	448.60



Out-of-Sample Validation

```

1 # --- Out-of-sample validation (last 30 business days) ---
2 test_len = 30
3 train_ts = ts.iloc[:-test_len] # all except last 30 obs
4 test_ts = ts.iloc[-test_len:] # last 30 obs
5
6 # Fit the same SARIMA specification on training data only
7 model_train = ARIMA(train_ts, order=(p,d,q), seasonal_order=(P,D,Q,s)).fit()
8
9 # 30-step-ahead forecast from the end of the training sample
10 forecast_test = model_train.get_forecast(steps=test_len, index=test_ts.index)
11 fc_mean = forecast_test.predicted_mean
12 fc_conf = forecast_test.conf_int(alpha=0.05)
13
14 # Align forecast index to test dates
15 fc_series = pd.Series(fc_mean.values, index=test_ts.index)

```

```

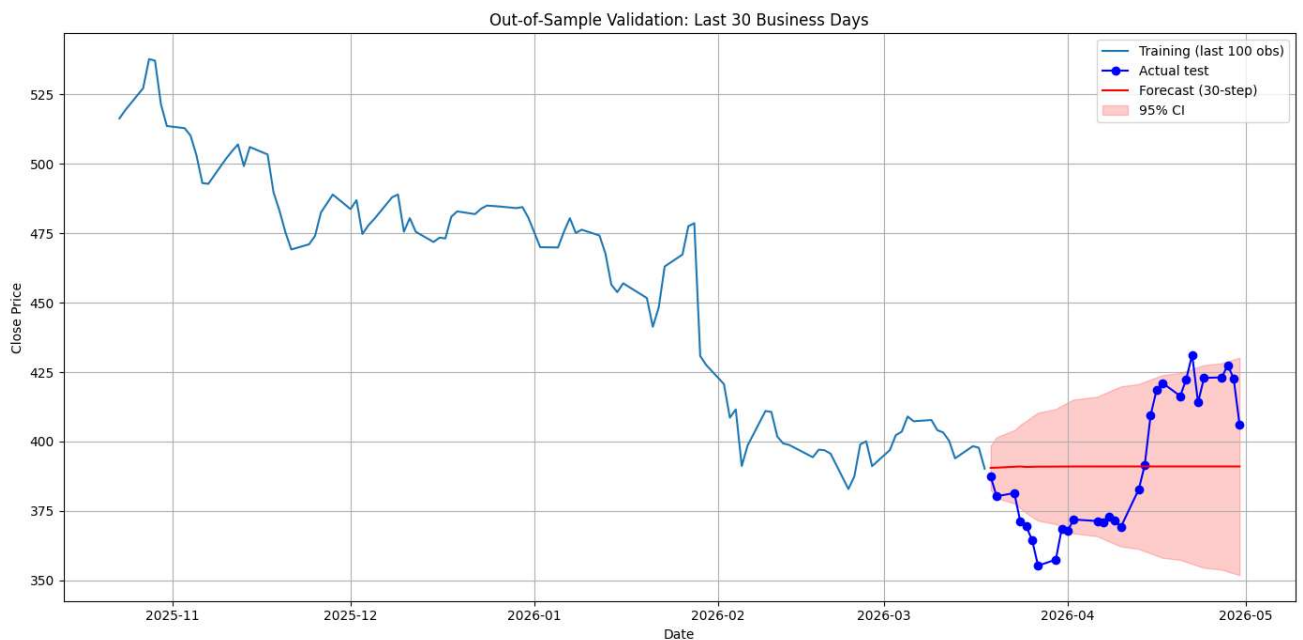
16
17 # Calculate error metrics
18 from sklearn.metrics import mean_absolute_error, mean_squared_error
19 import numpy as np
20 mae = mean_absolute_error(test_ts, fc_series)
21 rmse = np.sqrt(mean_squared_error(test_ts, fc_series))
22 print(f'Test MAE : {mae:.2f}')
23 print(f'Test RMSE: {rmse:.2f}')
24
25 # Plot actual vs. forecast
26 plt.figure(figsize=(14,7))
27 plt.plot(train_ts.index[-100:], train_ts[-100:], label='Training (last 100 obs)')
28 plt.plot(test_ts.index, test_ts, label='Actual test', color='blue', marker='o')
29 plt.plot(fc_series.index, fc_series, label='Forecast (30-step)', color='red')
30 plt.fill_between(test_ts.index, fc_conf.iloc[:,0], fc_conf.iloc[:,1],
31                 alpha=0.2, color='red', label='95% CI')
32 plt.title('Out-of-Sample Validation: Last 30 Business Days')
33 plt.xlabel('Date')
34 plt.ylabel('Close Price')
35 plt.legend()
36 plt.grid(True)
37 plt.tight_layout()
38 plt.show()

```

```

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided
self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided
self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided
self._init_dates(dates, freq)
Test MAE : 22.51
Test RMSE : 24.47

```



✓ PART 2: Multivariate Model (VAR or VECM)

✓ Prepare the Multivariate Data

```

1 # Create a DataFrame for multivariate analysis
2 mv_df = df[['MSFT', 'SP500', 'VIX']].copy()
3 mv_df.columns = ['MSFT', 'SP500', 'VIX']
4
5 print(mv_df.head())
6 print(mv_df.info())
7 print(mv_df.describe())

```

```

      MSFT      SP500      VIX
Date
2015-04-01  34.736565  2059.689941  15.11
2015-04-02  34.369770  2066.959961  14.67
2015-04-06  35.444607  2080.620117  14.74
2015-04-07  35.427551  2076.330078  14.78
2015-04-08  35.333706  2081.899902  13.98
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 2787 entries, 2015-04-01 to 2026-04-30
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ---
 0   MSFT    2787 non-null    float64
 1   SP500   2787 non-null    float64
 2   VIX     2787 non-null    float64
dtypes: float64(3)
memory usage: 87.1 KB
None
      MSFT      SP500      VIX
count  2787.000000  2787.000000  2787.000000
mean    214.649375   3743.954398   18.428281
std     143.901486   1410.020410    7.130172
min      34.369770   1829.079956    9.140000
25%      79.493378   2588.050049   13.595000
50%     202.800964   3443.439941   16.660000
75%     322.918488   4529.600098   21.375000
max     537.646423   7209.009766   82.690002

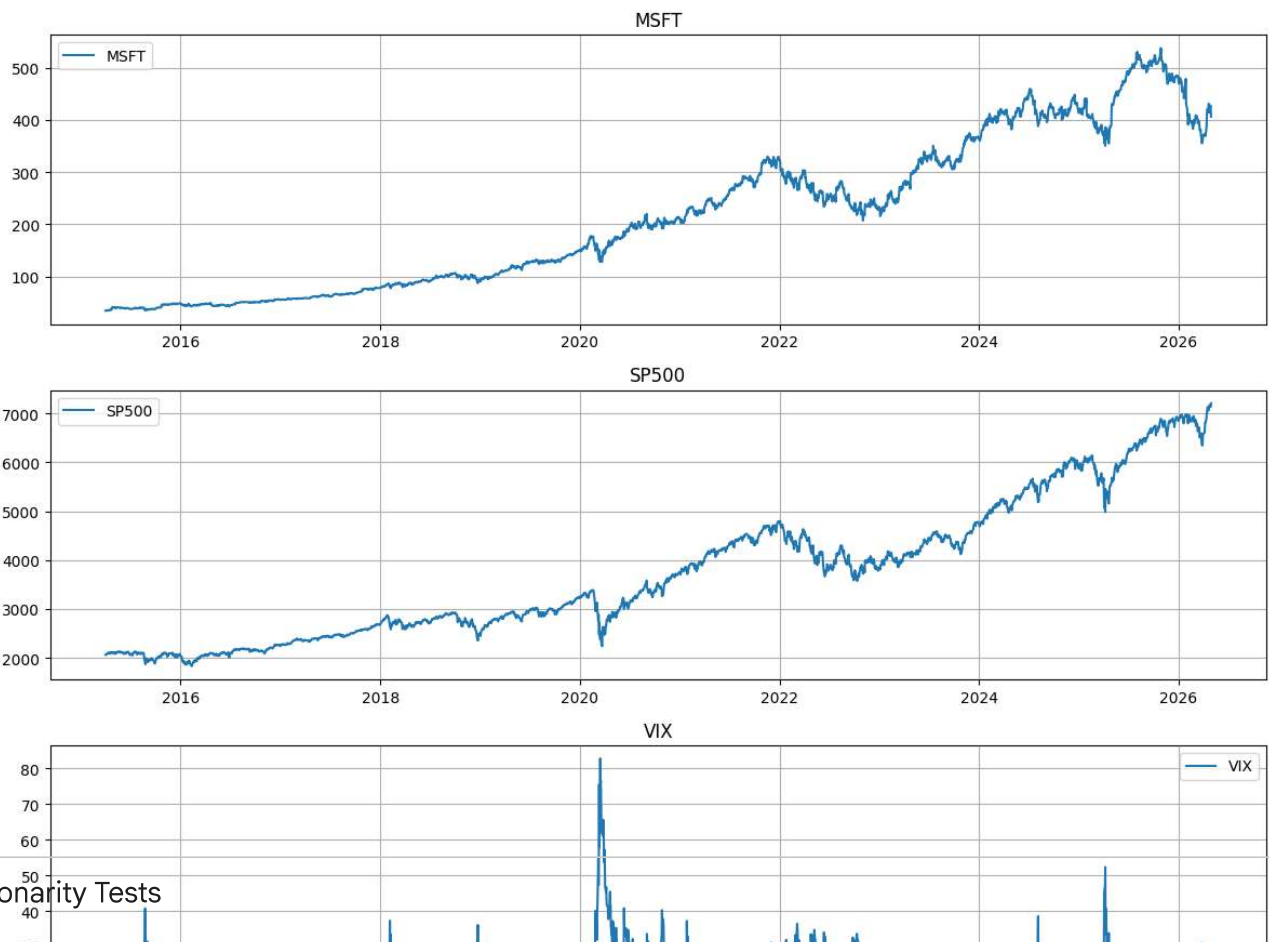
```

Visualise the Series and Their Relationships

```

1 import seaborn as sns
2
3 # Line plots
4 fig, axes = plt.subplots(3, 1, figsize=(12, 10))
5 for i, col in enumerate(mv_df.columns):
6     axes[i].plot(mv_df.index, mv_df[col], label=col)
7     axes[i].set_title(col)
8     axes[i].legend()
9     axes[i].grid(True)
10 plt.tight_layout()
11 plt.show()
12
13 # Scatter matrix
14 sns.pairplot(mv_df)
15 plt.show()
16
17 # Correlation matrix
18 corr = mv_df.corr()
19 sns.heatmap(corr, annot=True, cmap='coolwarm', center=0)
20 plt.title('Correlation Matrix')
21 plt.show()

```

Stationarity Tests

```

1 from statsmodels.tsa.stattools import adfuller, kpss
2
3 def stationarity_tests(series, title=""):
4     print(f"\n=== {title} ===")
5     adf = adfuller(series, autolag='AIC')
6     print(f'ADF Statistic: {adf[0]:.6f}, p-value: {adf[1]:.6f}')
7     kpss_stat, kpss_p, _, _ = kpss(series, regression='c')
8     print(f'KPSS Statistic: {kpss_stat:.6f}, p-value: {kpss_p:.6f}')
9
10 # Test levels and first differences
11 for col in mv_df.columns:
12     stationarity_tests(mv_df[col], f"Original {col}")
13     stationarity_tests(mv_df[col].diff().dropna(), f"First Difference of {col}")

```

```

=== Original MSFT ===
ADF Statistic: -0.609983, p-value: 0.868710
KPSS Statistic: 8.368368, p-value: 0.010000

=== First Difference of MSFT ===
/tmp/ipykernel_576/2459236321.py:7: InterpolationWarning: The test statistic is outside of the range of p-values available
  kpss_stat, kpss_p, _, _ = kpss(series, regression='c')
/tmp/ipykernel_576/2459236321.py:7: InterpolationWarning: The test statistic is outside of the range of p-values available
  kpss_stat, kpss_p, _, _ = kpss(series, regression='c')
ADF Statistic: -17.524786, p-value: 0.000000
KPSS Statistic: 0.063890, p-value: 0.100000

=== Original SP500 ===
ADF Statistic: 1.134508, p-value: 0.995501
KPSS Statistic: 8.008017, p-value: 0.010000

=== First Difference of SP500 ===
ADF Statistic: -17.307838, p-value: 0.000000
KPSS Statistic: 0.243493, p-value: 0.100000

=== Original VIX ===
/tmp/ipykernel_576/2459236321.py:7: InterpolationWarning: The test statistic is outside of the range of p-values available
  kpss_stat, kpss_p, _, _ = kpss(series, regression='c')
/tmp/ipykernel_576/2459236321.py:7: InterpolationWarning: The test statistic is outside of the range of p-values available
  kpss_stat, kpss_p, _, _ = kpss(series, regression='c')

```

```

kpss_stat, kpss_p, _ = kpss(series, regression='c')
ADF Statistic: -5.744969, p-value: 0.000001
KPSS Statistic: 0.982760, p-value: 0.010000

```

```

=== First Difference of VIX ===

```

```

ADF Statistic: -12.369789, p-value: 0.000000
KPSS Statistic: 0.008687, p-value: 0.100000

```

```

/tmp/ipykernel_576/2459236321.py:7: InterpolationWarning: The test statistic is outside of the range of p-values available

```

```

kpss_stat, kpss_p, _ = kpss(series, regression='c')

```

```

/tmp/ipykernel_576/2459236321.py:7: InterpolationWarning: The test statistic is outside of the range of p-values available

```

```

kpss_stat, kpss_p, _ = kpss(series, regression='c')

```



✧ Cointegration Test (Johansen)

```

1 from statsmodels.tsa.vector_ar.vecm import coint_johansen
2 lags = 5
3
4 def johansen_test(df, lags):
5     """Perform Johansen cointegration test"""
6     res = coint_johansen(df, det_order=0, k_ar_diff=lags)
7     # Trace statistic
8     trace_stat = res.lr1
9     trace_crit = res.cvt[:, 1] # 95% critical value
10    # Max-eigen statistic
11    max_eigen_stat = res.lr2
12    max_eigen_crit = res.cvm[:, 1]
13
14    print("\n=== Johansen Trace Test ===")
15    for i in range(len(trace_stat)):
16        print(f"H0: r<={i} | Trace Stat: {trace_stat[i]:.3f} | 95% crit: {trace_crit[i]:.3f}")
17
18    print("\n=== Johansen Max-Eigen Test ===")
19    for i in range(len(max_eigen_stat)):
20        print(f"H0: r={i} | Max-Eigen Stat: {max_eigen_stat[i]:.3f} | 95% crit: {max_eigen_crit[i]:.3f}")
21
22 johansen_test(mv_df, lags=lags)

```

```

=== Johansen Trace Test ===

```

```

H0: r<=0 | Trace Stat: 78.391 | 95% crit: 29.796

```

```

H0: r<=1 | Trace Stat: 10.137 | 95% crit: 15.494

```

```

H0: r<=2 | Trace Stat: 2.219 | 95% crit: 3.841

```

```

=== Johansen Max-Eigen Test ===

```

```

H0: r=0 | Max-Eigen Stat: 68.254 | 95% crit: 21.131

```

```

H0: r=1 | Max-Eigen Stat: 7.917 | 95% crit: 14.264

```

```

H0: r=2 | Max-Eigen Stat: 2.219 | 95% crit: 3.841

```

✧ Model Order Selection (VAR/VECM)

```

1 from statsmodels.tsa.api import VAR
2
3 # Determine optimal lag order for a VAR model (in levels, but for VECM we use the same lags of differenced terms)
4 model_var = VAR(mv_df.diff().dropna())
5 lag_order = model_var.select_order(maxlags=10)
6 print(lag_order.summary())

```

```

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided
self._init_dates(dates, freq)

```

```

VAR Order Selection (* highlights the minimums)

```

	AIC	BIC	FPE	HQIC
0	10.10	10.11	2.441e+04	10.11
1	10.08	10.10*	2.379e+04	10.09
2	10.07	10.11	2.360e+04	10.09
3	10.07	10.13	2.353e+04	10.09
4	10.05	10.13	2.317e+04	10.08*
5	10.05	10.15	2.314e+04	10.09
6	10.05	10.17	2.314e+04	10.09
7	10.04	10.19	2.303e+04	10.10

8	10.04	10.20	2.300e+04	10.10
9	10.04	10.22	2.288e+04	10.10
10	10.04*	10.23	2.284e+04*	10.11

▼ Estimate the VECM (Vector Error Correction Model)

```

1  # --- Test which lag eliminates residual autocorrelation ---
2  from statsmodels.tsa.vector_ar.vecm import VECM
3  from statsmodels.stats.diagnostic import acorr_ljungbox
4
5  best_lag = None
6  for k_ar_diff in [5,6,7,8]:
7      vecm_test = VECM(mv_df, k_ar_diff=k_ar_diff, coint_rank=1,
8                      deterministic='ci')
9      vecm_test_res = vecm_test.fit()
10     residuals = pd.DataFrame(vecm_test_res.resid, columns=mv_df.columns)
11     all_ok = True
12     print(f"\n--- k_ar_diff = {k_ar_diff} ---")
13     for col in residuals.columns:
14         lb = acorr_ljungbox(residuals[col].dropna(), lags=[10], return_df=True)
15         pval = lb['lb_pvalue'].values[0]
16         status = "OK" if pval > 0.05 else "FAIL"
17         print(f" {col} p-value (lag 10) = {pval:.4f} {status}")
18         if pval <= 0.05:
19             all_ok = False
20     if all_ok:
21         best_lag = k_ar_diff
22         break
23     print(f"\n** Suggested k_ar_diff for white-noise residuals: {best_lag} **")

```

--- k_ar_diff = 5 ---

MSFT p-value (lag 10) = 0.2770 OK
 SP500 p-value (lag 10) = 0.0072 FAIL
 VIX p-value (lag 10) = 0.0178 FAIL

--- k_ar_diff = 6 ---

MSFT p-value (lag 10) = 0.6119 OK
 SP500 p-value (lag 10) = 0.0184 FAIL
 VIX p-value (lag 10) = 0.0274 FAIL

--- k_ar_diff = 7 ---

MSFT p-value (lag 10) = 0.6409 OK
 SP500 p-value (lag 10) = 0.0871 OK
 VIX p-value (lag 10) = 0.0332 FAIL

--- k_ar_diff = 8 ---

MSFT p-value (lag 10) = 0.8544 OK
 SP500 p-value (lag 10) = 0.4427 OK
 VIX p-value (lag 10) = 0.7341 OK

** Suggested k_ar_diff for white-noise residuals: 8 **

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided but will not be used, as the data is not indexed by date.
 self._init_dates(dates, freq)
 /usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided but will not be used, as the data is not indexed by date.
 self._init_dates(dates, freq)
 /usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided but will not be used, as the data is not indexed by date.
 self._init_dates(dates, freq)
 /usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided but will not be used, as the data is not indexed by date.
 self._init_dates(dates, freq)

```

1  from statsmodels.tsa.api import VAR
2  from statsmodels.tsa.vector_ar.vecm import VECM
3
4  # --- Estimate VECM with chosen lag ---
5  k_ar_diff = 8 # lag selected by the residual autocorrelation check above
6  r = 1
7  vecm = VECM(mv_df, k_ar_diff=k_ar_diff, coint_rank=r, deterministic='ci')
8  vecm_res = vecm.fit()
9  print(vecm_res.summary())

```

```

L8.MSFT      -0.0118    0.274    -0.043    0.966    -0.548    0.525
L8.SP500     0.0133    0.037    0.362    0.718    -0.059    0.085
L8.VIX       1.6642    0.641    2.598    0.009    0.409    2.920
Det. terms outside the coint. relation & lagged endog. parameters for equation VIX
=====
      coef      std err      z      P>|z|      [0.025      0.975]
-----
L1.MSFT      0.0195    0.013    1.550    0.121    -0.005    0.044
L1.SP500     -0.0027    0.002   -1.559    0.119    -0.006    0.001
L1.VIX       -0.1725    0.030   -5.810    0.000   -0.231   -0.114
L2.MSFT      0.0154    0.013    1.230    0.219   -0.009    0.040
L2.SP500     0.0002    0.002    0.120    0.905   -0.003    0.004
L2.VIX       0.0363    0.030    1.215    0.224   -0.022    0.095
L3.MSFT      -0.0070    0.013   -0.555    0.579   -0.032    0.018
L3.SP500     0.0036    0.002    2.096    0.036    0.000    0.007
L3.VIX       0.0592    0.030    1.983    0.047    0.001    0.118
L4.MSFT      -0.0014    0.013   -0.115    0.909   -0.026    0.023
L4.SP500     -0.0021    0.002   -1.213    0.225   -0.005    0.001
L4.VIX       -0.1046    0.030   -3.504    0.000   -0.163   -0.046
L5.MSFT      0.0063    0.013    0.501    0.616   -0.018    0.031
L5.SP500     -0.0005    0.002   -0.294    0.769   -0.004    0.003
L5.VIX       0.0017    0.030    0.058    0.954   -0.057    0.060
L6.MSFT      -0.0107    0.013   -0.850    0.395   -0.035    0.014
L6.SP500     0.0019    0.002    1.098    0.272   -0.001    0.005
L6.VIX       -0.0044    0.030   -0.146    0.884   -0.063    0.054
L7.MSFT      -0.0134    0.013   -1.064    0.287   -0.038    0.011
L7.SP500     -0.0013    0.002   -0.749    0.454   -0.005    0.002
L7.VIX       -0.0441    0.030   -1.478    0.139   -0.103    0.014
L8.MSFT      -0.0059    0.013   -0.468    0.640   -0.031    0.019
L8.SP500     0.0011    0.002    0.651    0.515   -0.002    0.004
L8.VIX       -0.0579    0.030   -1.956    0.051   -0.116    0.000
Loading coefficients (alpha) for equation MSFT
=====
      coef      std err      z      P>|z|      [0.025      0.975]
-----
ec1          -0.0002    0.000   -0.865    0.387   -0.001    0.000
Loading coefficients (alpha) for equation SP500
=====
      coef      std err      z      P>|z|      [0.025      0.975]
-----
ec1           0.0010    0.002    0.472    0.637   -0.003    0.005
Loading coefficients (alpha) for equation VIX
=====
      coef      std err      z      P>|z|      [0.025      0.975]
-----
ec1           0.0004    0.000    4.440    0.000    0.000    0.001
Cointegration relations for loading-coefficients-column 1
=====
      coef      std err      z      P>|z|      [0.025      0.975]
-----
beta.1        1.0000         0         0      0.000    1.000    1.000
beta.2         0.0289    0.033    0.876    0.381   -0.036    0.094
beta.3       -47.8805    6.935   -6.905    0.000   -61.472   -34.289
const        715.6417   172.125    4.158    0.000   378.283   1053.001
=====
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been pr
self._init_dates(dates, freq)

```

Diagnostic Checks

```

1 # Residual autocorrelation test (Portmanteau)
2 from statsmodels.stats.diagnostic import acorr_ljungbox
3 import pandas as pd # Import pandas
4
5 residuals = pd.DataFrame(vecm_res.resid, columns=mv_df.columns) # Convert to
Dataframe with original column names
6 for col in residuals.columns:
7     lb = acorr_ljungbox(residuals[col].dropna(), lags=[10], return_df=True)
8     print(f"\nLjung-Box for {col} residuals:")
9     print(lb)

```

Ljung-Box for MSFT residuals:

```

      lb_stat  lb_pvalue
10  5.512913  0.854393

```

Ljung-Box for SP500 residuals:

	lb_stat	lb_pvalue
10	9.97487	0.442701

Ljung-Box for VIX residuals:

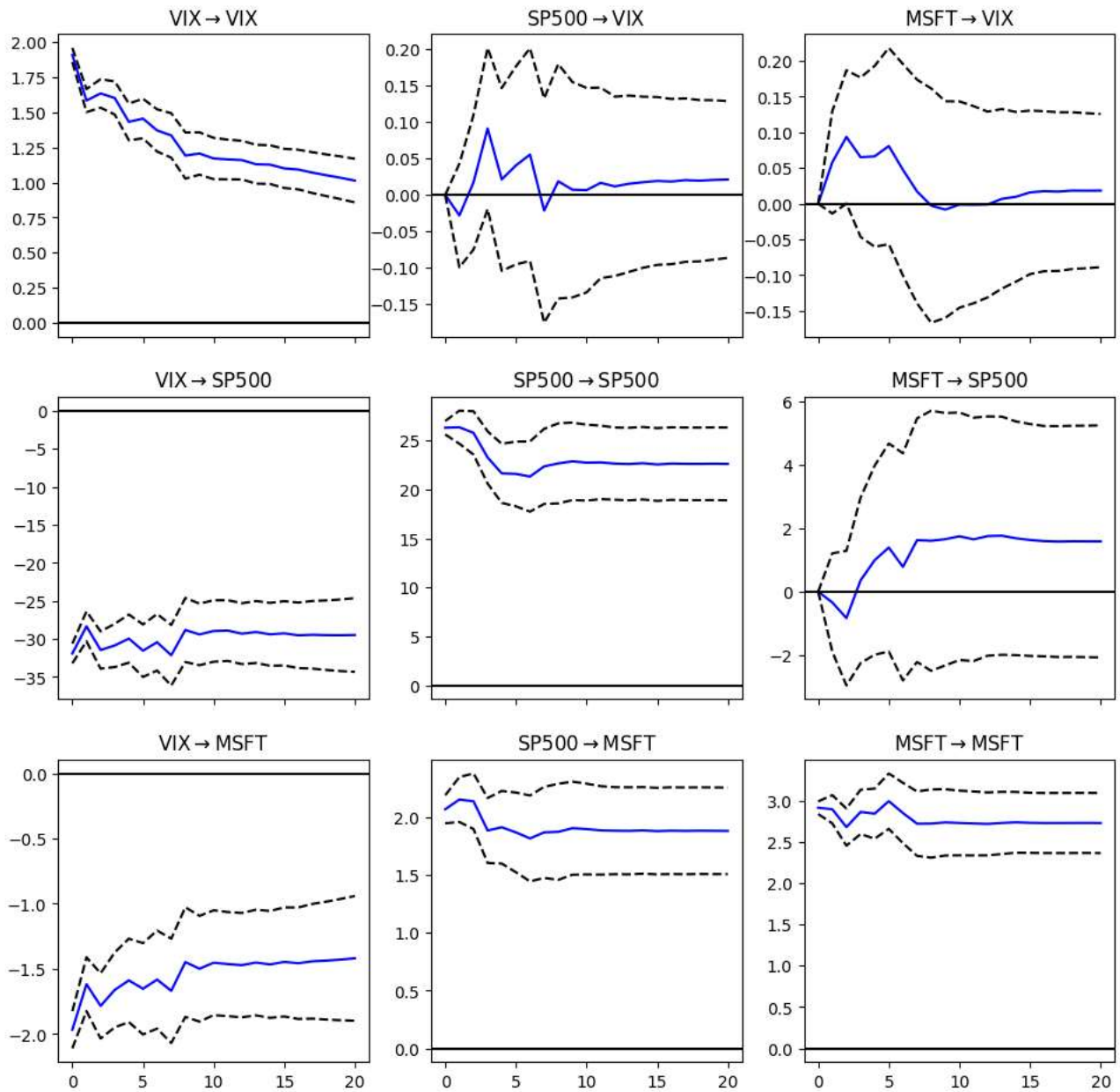
	lb_stat	lb_pvalue
10	6.908294	0.734073

✓ Impulse Response Functions

```
1 # Reorder columns for a meaningful causal ordering: VIX -> SP500 -> MSFT
2 mv_df_reorder = mv_df[['VIX', 'SP500', 'MSFT']].copy()
3 vecm_reorder = VECM(mv_df_reorder, k_ar_diff=k_ar_diff, coint_rank=r,
4   deterministic='ci')
5 vecm_reorder_res = vecm_reorder.fit()
6 irf_reorder = vecm_reorder_res.irf(periods=20)
7 irf_reorder.plot(orth=True)
8 plt.show()
```

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been pr
self._init_dates(dates, freq)

Impulse responses (orthogonalized)



Forecasting

```

1 forecast_steps = 30
2 forecast, lower, upper = vecm_res.predict(forecast_steps, 0.05)
3
4 # Convert to DataFrame for the first variable (e.g., MSFT)
5 forecast_df = pd.DataFrame({
6     'MSFT_forecast': forecast[:, 0],
7     'lower_CI': lower[:, 0],
8     'upper_CI': upper[:, 0]
9 }, index=pd.date_range(start=mv_df.index[-1] + pd.Timedelta(days=1),
10                        periods=forecast_steps, freq='B'))
11
12 print(forecast_df.head())
13 # Plot MSFT forecast

```

```

14 plt.figure(figsize=(14,7))
15 plt.plot(mv_df.index[-100:], mv_df['MSFT'].iloc[-100:], label='Historical MSFT')
16 plt.plot(forecast_df.index, forecast_df['MSFT_forecast'], label='VECM
Forecast', color='red')
17 plt.fill_between(forecast_df.index, forecast_df['lower_CI'], forecast_df
['upper_CI'], alpha=0.2, color='red')
18 plt.title('MSFT Stock Price Forecast (VECM)')
19 plt.legend()
20 plt.grid(True)
21 plt.show()

```

	MSFT_forecast	lower_CI	upper_CI
2026-05-01	408.057895	400.067760	416.048030
2026-05-04	409.342723	398.215627	420.469818
2026-05-05	408.014687	394.558108	421.471266
2026-05-06	407.374880	391.988828	422.760932
2026-05-07	406.085428	389.014206	423.156650



Out-of-sample validation

```

1 # --- Out-of-sample validation for VECM ---
2 test_len = 30
3 train_df = mv_df.iloc[:-test_len]
4 test_df = mv_df.iloc[-test_len:]
5
6 # Fit VECM on training data (use the same k_ar_diff and r as above)
7 vecm_train = VECM(train_df, k_ar_diff=k_ar_diff, coint_rank=r,
deterministic='ci')
8 vecm_res_train = vecm_train.fit()
9
10 # Forecast test_len steps ahead
11 forecast_array = vecm_res_train.predict(steps=test_len)
12
13 # Convert the forecast NumPy array to a DataFrame with correct column names and
index
14 forecast = pd.DataFrame(forecast_array, columns=mv_df.columns, index=test_df.
index)
15
16 from sklearn.metrics import mean_squared_error
17 import numpy as np
18
19 for col in mv_df.columns:
20     rmse = np.sqrt(mean_squared_error(test_df[col], forecast[col]))
21     mae = mean_absolute_error(test_df[col], forecast[col])
22     print(f"{col} - RMSE: {rmse:.2f}, MAE: {mae:.2f}")
23
24 # Optional: plot actual vs forecast for one variable

```

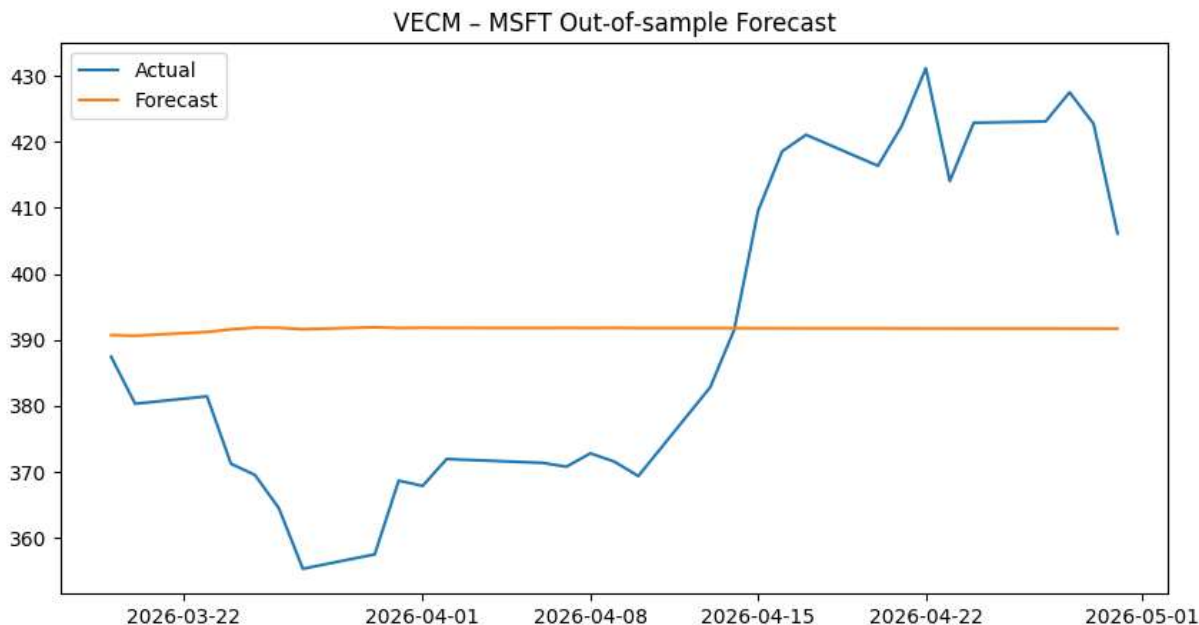
```

25 import matplotlib.pyplot as plt
26 plt.figure(figsize=(10,5))
27 plt.plot(test_df.index, test_df['MSFT'], label='Actual')
28 plt.plot(forecast.index, forecast['MSFT'], label='Forecast')
29 plt.legend(); plt.title('VECM - MSFT Out-of-sample Forecast'); plt.show()

```

/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: A date index has been provided to the model, but it is not used in the estimation. This may lead to incorrect results.
 self._init_dates(dates, freq)

MSFT - RMSE: 24.49, MAE: 22.60
 SP500 - RMSE: 341.47, MAE: 278.03
 VIX - RMSE: 3.96, MAE: 3.46



✓ PART 3: Conditional Heteroscedasticity (ARCH/GARCH)

```

1 import sys
2 !{sys.executable} -m pip install arch --quiet

```

982.9/982.9 kB 11.4 MB/s eta 0:00:00

✓ Test ARIMA Residuals for ARCH Effects

```

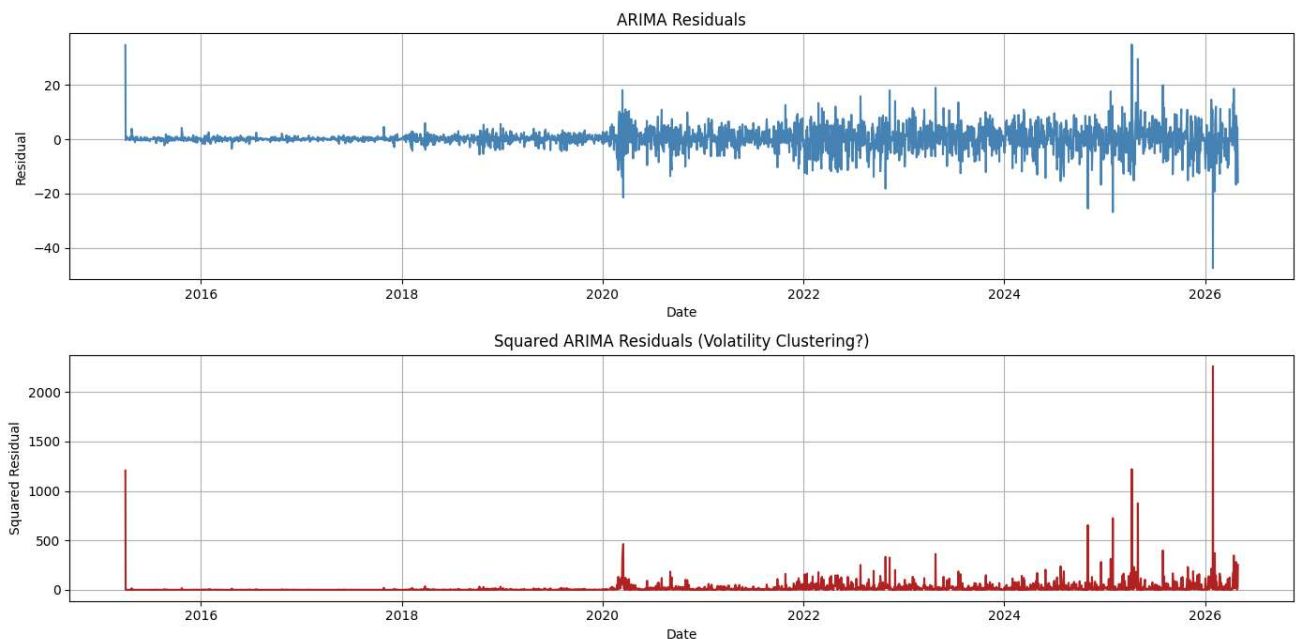
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from arch.unitroot import PhillipsPerron
5 from statsmodels.stats.diagnostic import het_arch
6 from arch import arch_model
7
8 # — Retrieve residuals from the Part 1 ARIMA model —
9 arima_resid = arima_model.resid.dropna()
10
11 # Plot residuals and squared residuals
12 fig, axes = plt.subplots(2, 1, figsize=(14, 7))
13 axes[0].plot(arima_resid, color='steelblue')
14 axes[0].set_title('ARIMA Residuals')
15 axes[0].set_xlabel('Date')
16 axes[0].set_ylabel('Residual')
17 axes[0].grid(True)
18
19 axes[1].plot(arima_resid**2, color='firebrick')
20 axes[1].set_title('Squared ARIMA Residuals (Volatility Clustering?)')
21 axes[1].set_xlabel('Date')
22 axes[1].set_ylabel('Squared Residual')
23 axes[1].grid(True)
24

```

```

25 plt.tight_layout()
26 plt.show()
27
28 # — ARCH-LM Test on ARIMA residuals —
29 print("=" * 55)
30 print("ARCH-LM Test on ARIMA Residuals")
31 print("=" * 55)
32 for lag in [5, 10, 15]:
33     lm_stat, lm_pval, f_stat, f_pval = het_arch(arima_resid, nlags=lag)
34     result = "ARCH effects present X" if lm_pval < 0.05 else "No ARCH effects ✓"
35     print(f"Lags={lag:2d} | LM Stat: {lm_stat:.4f} | p-value: {lm_pval:.6f} | {result}")

```



=====

ARCH-LM Test on ARIMA Residuals

=====

```

Lags= 5 | LM Stat: 123.1704 | p-value: 0.000000 | ARCH effects present X
Lags=10 | LM Stat: 156.1968 | p-value: 0.000000 | ARCH effects present X
Lags=15 | LM Stat: 183.6884 | p-value: 0.000000 | ARCH effects present X

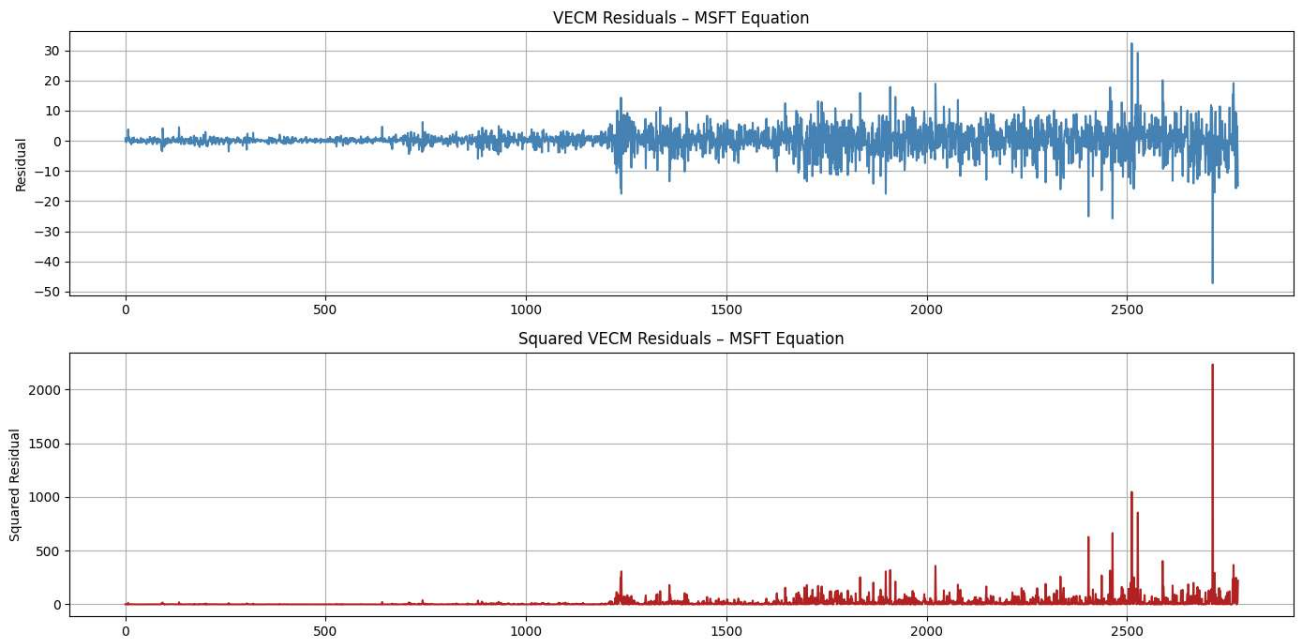
```

✓ Test VECM Residuals for ARCH Effects

```

1 # — ARCH-LM Test on VECM residuals (MSFT equation only) —
2 vecm_resid_msft = pd.Series(vecm_res.resid[:, mv_df.columns.get_loc('MSFT')],
3                             name='VECM_MSFT_resid')
4
5 fig, axes = plt.subplots(2, 1, figsize=(14, 7))
6 axes[0].plot(vecm_resid_msft.values, color='steelblue')
7 axes[0].set_title('VECM Residuals - MSFT Equation')
8 axes[0].set_ylabel('Residual')
9 axes[0].grid(True)
10
11 axes[1].plot(vecm_resid_msft.values**2, color='firebrick')
12 axes[1].set_title('Squared VECM Residuals - MSFT Equation')
13 axes[1].set_ylabel('Squared Residual')
14 axes[1].grid(True)
15
16 plt.tight_layout()
17 plt.show()
18
19 print("=" * 55)
20 print("ARCH-LM Test on VECM Residuals (MSFT equation)")
21 print("=" * 55)
22 for lag in [5, 10, 15]:
23     lm_stat, lm_pval, f_stat, f_pval = het_arch(vecm_resid_msft.dropna(), nlags=lag)
24     result = "ARCH effects present X" if lm_pval < 0.05 else "No ARCH effects ✓"
25     print(f"Lags={lag:2d} | LM Stat: {lm_stat:.4f} | p-value: {lm_pval:.6f} | {result}")

```



```
=====
ARCH-LM Test on VECM Residuals (MSFT equation)
=====
Lags= 5 | LM Stat: 111.1921 | p-value: 0.000000 | ARCH effects present X
Lags=10 | LM Stat: 141.4155 | p-value: 0.000000 | ARCH effects present X
Lags=15 | LM Stat: 167.9833 | p-value: 0.000000 | ARCH effects present X
```

✓ Fit GARCH(1,1) on ARIMA Residuals

```
1 # — GARCH(1,1) on ARIMA residuals —
2 print("=" * 55)
3 print("GARCH(1,1) - ARIMA Residuals")
4 print("=" * 55)
5
6 garch_arima = arch_model(arima_resid, vol='Garch', p=1, q=1, dist='normal')
7 garch_arima_res = garch_arima.fit(dis='off')
8 print(garch_arima_res.summary())
```

```
=====
GARCH(1,1) - ARIMA Residuals
=====
Constant Mean - GARCH Model Results
=====
Dep. Variable:          None    R-squared:                0.000
Mean Model:            Constant Mean    Adj. R-squared:          0.000
Vol Model:             GARCH    Log-Likelihood:         -6596.60
Distribution:          Normal    AIC:                   13201.2
Method:               Maximum Likelihood    BIC:                   13224.9
No. Observations:      2787
Date:                 Thu, Oct 08 2026    Df Residuals:           2786
Time:                 08:48:23            Df Model:                1
Mean Model
=====
coef    std err        t    P>|t|    95.0% Conf. Int.
-----
mu      0.0887    2.326e-02    3.815    1.360e-04    [4.315e-02, 0.134]
Volatility Model
=====
coef    std err        t    P>|t|    95.0% Conf. Int.
-----
omega    0.0242    1.083e-02    2.237    2.528e-02    [3.000e-03, 4.544e-02]
alpha[1] 0.1159    1.702e-02    6.809    9.805e-12    [8.255e-02, 0.149]
beta[1]  0.8841    1.790e-02    49.400    0.000        [0.849, 0.919]
=====
Covariance estimator: robust
```

Model Selection: Compare GARCH Variants

```

1 # — Compare GARCH variants to select best model —
2 results_table = []
3
4 specs = [
5     ('GARCH(1,1)', dict(vol='Garch', p=1, q=1, dist='normal')),
6     ('GARCH(1,1)-t', dict(vol='Garch', p=1, q=1, dist='t')),
7     ('EGARCH(1,1)', dict(vol='EGARCH', p=1, q=1, dist='normal')),
8     ('GJR-GARCH', dict(vol='GARCH', p=1, o=1, q=1, dist='normal')),
9 ]
10
11 print(f"{'Model':<20} {'AIC':>10} {'BIC':>10} {'Log-Lik':>12}")
12 print("-" * 55)
13 for name, kwargs in specs:
14     try:
15         m = arch_model(arima_resid, **kwargs).fit(dispen='off')
16         print(f"{'name':<20} {m.aic:>10.2f} {m.bic:>10.2f} {m.loglikelihood:>12.2f}")
17         results_table.append((name, m.aic, m.bic))
18     except Exception as e:
19         print(f"{'name':<20} Failed: {e}")
20
21 best_model_name = min(results_table, key=lambda x: x[1])[0]
22 print(f"\n✓ Best model by AIC: {best_model_name}")

```

Model	AIC	BIC	Log-Lik
GARCH(1,1)	13201.20	13224.93	-6596.60
GARCH(1,1)-t	12810.89	12840.55	-6400.44
EGARCH(1,1)	13141.47	13165.20	-6566.73
GJR-GARCH	13203.17	13232.83	-6596.58

✓ Best model by AIC: GARCH(1,1)-t

Fit Best GARCH Model & Diagnostics

```

1 # — Fit best model (GARCH(1,1)-t had the lowest AIC above) —
2 best_garch = arch_model(arima_resid, vol='Garch', p=1, q=1, dist='t')
3 best_garch_res = best_garch.fit(dispen='off')
4 print(best_garch_res.summary())
5
6 # — Parameter interpretation —
7 params = best_garch_res.params
8 alpha = params['alpha[1]']
9 beta = params['beta[1]']
10 print(f"nomega = {params['omega']:.6f}")
11 print(f"alpha1 = {alpha:.4f} (ARCH effect - reaction to shocks)")
12 print(f"beta1 = {beta:.4f} (GARCH effect - volatility persistence)")
13 print(f"alpha1 + beta1 = {alpha+beta:.4f} (persistence; <1 required for stationarity)")

```

Constant Mean - GARCH Model Results

```

=====
Dep. Variable:          None    R-squared:                0.000
Mean Model:             Constant Mean    Adj. R-squared:          0.000
Vol Model:              GARCH          Log-Likelihood:        -6400.44
Distribution:           Standardized Student's t    AIC:                  12810.9
Method:                 Maximum Likelihood    BIC:                  12840.6
                                     No. Observations:      2787
Date:                   Thu, Oct 08 2026    Df Residuals:         2786
Time:                   08:48:24           Df Model:              1
                                     Mean Model
=====

```

	coef	std err	t	P> t	95.0% Conf. Int.
mu	0.0947	1.845e-02	5.133	2.844e-07	[5.854e-02, 0.131]

Volatility Model

```

=====

```

	coef	std err	t	P> t	95.0% Conf. Int.
omega	0.0122	4.897e-03	2.491	1.273e-02	[2.602e-03, 2.180e-02]
alpha[1]	0.1343	1.178e-02	11.402	4.099e-30	[0.111, 0.157]

```

beta[1]      0.8657  1.330e-02   65.106    0.000    [ 0.840,  0.892]
              Distribution
=====
              coef      std err          t      P>|t|   95.0% Conf. Int.
-----
nu           5.0656      0.366     13.853  1.214e-43 [ 4.349,  5.782]
=====

```

Covariance estimator: robust

```

omega  = 0.012201
alpha1 = 0.1343  (ARCH effect - reaction to shocks)
beta1  = 0.8657  (GARCH effect - volatility persistence)
alpha1 + beta1 = 1.0000 (persistence; <1 required for stationarity)

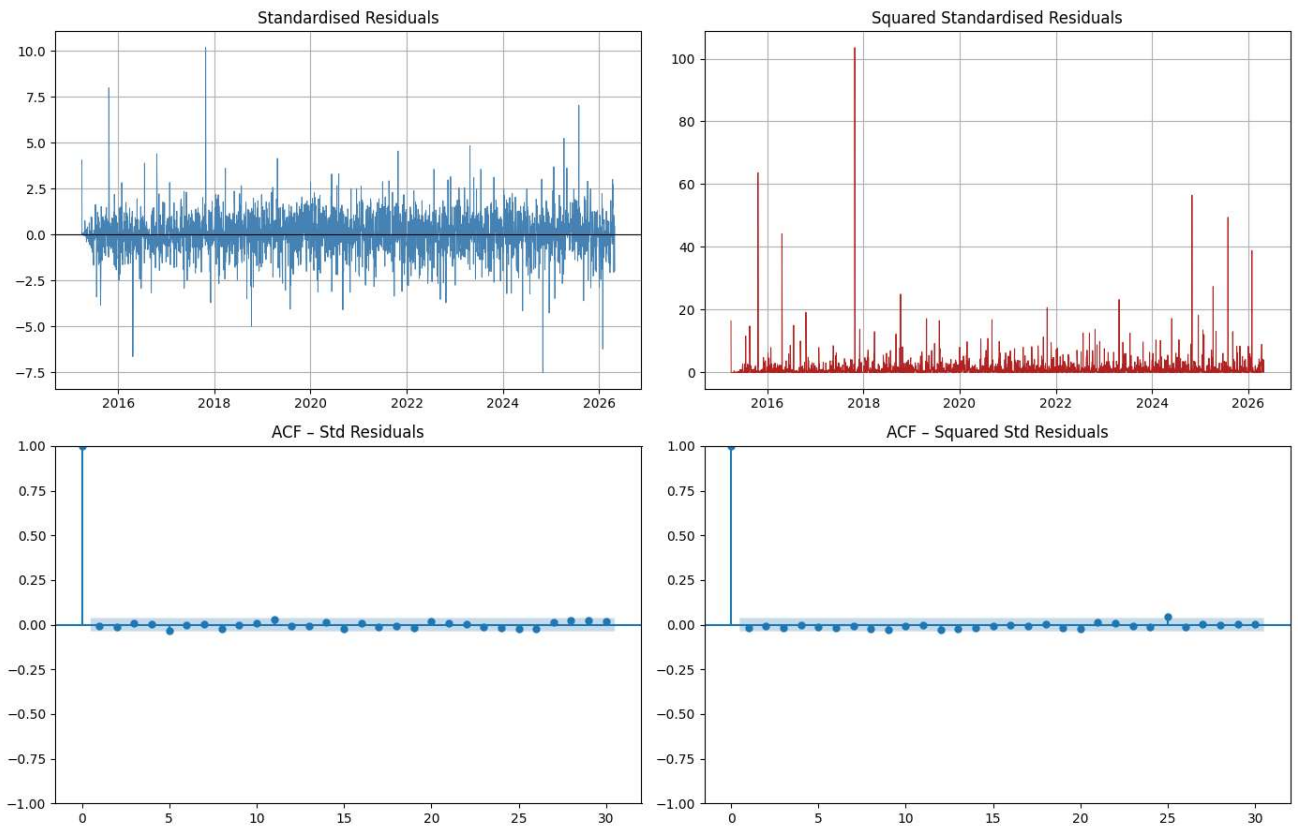
```

✓ GARCH Diagnostics (Standardised Residuals)

```

1 # — Standardised residual diagnostics —
2 std_resid = best_garch_res.std_resid
3
4 fig, axes = plt.subplots(2, 2, figsize=(14, 9))
5
6 # 1. Standardised residuals over time
7 axes[0,0].plot(std_resid, color='steelblue', linewidth=0.7)
8 axes[0,0].axhline(0, color='black', linewidth=0.8)
9 axes[0,0].set_title('Standardised Residuals')
10 axes[0,0].grid(True)
11
12 # 2. Squared standardised residuals
13 axes[0,1].plot(std_resid**2, color='firebrick', linewidth=0.7)
14 axes[0,1].set_title('Squared Standardised Residuals')
15 axes[0,1].grid(True)
16
17 # 3. ACF of standardised residuals
18 from statsmodels.graphics.tsaplots import plot_acf
19 plot_acf(std_resid, lags=30, ax=axes[1,0], title='ACF - Std Residuals')
20
21 # 4. ACF of squared standardised residuals
22 plot_acf(std_resid**2, lags=30, ax=axes[1,1], title='ACF - Squared Std Residuals')
23
24 plt.tight_layout()
25 plt.show()
26
27 # — Ljung-Box on standardised and squared standardised residuals —
28 from statsmodels.stats.diagnostic import acorr_ljungbox
29 print("Ljung-Box on Standardised Residuals (should be non-significant):")
30 print(acorr_ljungbox(std_resid, lags=[10, 20], return_df=True))
31
32 print("\nLjung-Box on SQUARED Standardised Residuals (should be non-significant):")
33 print(acorr_ljungbox(std_resid**2, lags=[10, 20], return_df=True))
34
35 # — ARCH-LM test on standardised residuals (confirm ARCH removed) —
36 print("\nARCH-LM Test on Standardised Residuals (confirm ARCH effects removed):")
37 for lag in [5, 10]:
38     lm_stat, lm_pval, _, _ = het_arch(std_resid, nlags=lag)
39     result = "ARCH effects remain X" if lm_pval < 0.05 else "ARCH effects removed ✓"
40     print(f"Lags={lag:2d} | p-value: {lm_pval:.6f} | {result}")

```



Ljung-Box on Standardised Residuals (should be non-significant):

	lb_stat	lb_pvalue
10	5.614836	0.846519
20	12.654042	0.891729

Ljung-Box on SQUARED Standardised Residuals (should be non-significant):

	lb_stat	lb_pvalue
10	7.902088	0.638400
20	15.927751	0.721096

ARCH-LM Test on Standardised Residuals (confirm ARCH effects removed):

Lags= 5 | p-value: 0.752138 | ARCH effects removed ✓
 Lags=10 | p-value: 0.581250 | ARCH effects removed ✓

Conditional Volatility Plot

```

1 # — Plot conditional volatility (the estimated  $\sigma_t$ ) —
2 cond_vol = best_garch_res.conditional_volatility
3
4 plt.figure(figsize=(14, 6))
5 plt.plot(arima_resid.index, cond_vol, color='darkorange', linewidth=1.2, label='Conditional Volatility  $\sigma_t$ ')
6 plt.axvline(pd.Timestamp('2020-03-01'), color='red', linestyle='--', linewidth=1.5, label='COVID-19 Break')
7 plt.title('GARCH(1,1) Conditional Volatility - MSFT ARIMA Residuals')
8 plt.xlabel('Date')
9 plt.ylabel('Volatility ( $\sigma_t$ )')
10 plt.legend()
11 plt.grid(True)
12 plt.tight_layout()
13 plt.show()

```